# MLDL Practical 9

**Name: Ishaan Khan**
**Class: D15C**
**Roll No: 29**
**Batch: B**

---

**Aim:** Implement a Recurrent Neural Network (RNN) / LSTM for text data (IMDB Movie Reviews)

## Dataset Source

**Dataset Name:** IMDB Movie Reviews Dataset

**Platform:** TensorFlow Keras (built-in)

**Dataset Link:** tensorflow.keras.datasets.imdb

## Dataset Description

The IMDB Movie Reviews dataset is a widely-used benchmark for binary sentiment classification in Natural Language Processing (NLP). It consists of 50,000 movie reviews equally split into positive and negative categories, making it a balanced binary classification problem well-suited for training LSTM-based sequence models.

### Dataset Characteristics

- **Total Reviews:** 50,000 (25,000 training + 25,000 testing)
- **Number of Classes:** 2 — 0 = Negative Sentiment, 1 = Positive Sentiment
- **Vocabulary Size:** Top 10,000 most frequent words considered (MAX_FEATURES = 10,000)
- **Sequence Length:** All sequences padded or truncated to 200 tokens (MAX_LEN = 200)
- **Data Format:** Integer-encoded word sequences

| Column Name | Data Type | Description |
|---|---|---|
| Review (Text) | Integer Sequence | Word indices representing tokenized review content |
| Label | Binary (0 or 1) | 0 = Negative Sentiment, 1 = Positive Sentiment |

**Dataset Source:** https://keras.io/api/datasets/imdb/

## Recurrent Neural Network (RNN) / LSTM – Theory

Recurrent Neural Networks (RNNs) are a class of deep learning models designed for processing sequential data such as text, speech, and time-series signals. Unlike feedforward neural networks, RNNs maintain a hidden state that captures information from previous time steps, enabling them to model temporal dependencies.

Given an input sequence $X \in \mathbb{R}^{(T \times d)}$, where T is the sequence length and d is the embedding dimension, the RNN updates its hidden state at each time step t as:

$$\mathbf{h_t = \tanh(W_x \cdot x_t + W_h \cdot h_{t-1} + b)}$$

Standard RNNs suffer from the vanishing gradient problem, which makes learning long-term dependencies difficult. Long Short-Term Memory (LSTM) networks overcome this using gating mechanisms — an input gate, forget gate, and output gate — that control the flow of information through a memory cell, enabling learning of long-range contextual patterns in text.

## Algorithm Limitations

- **Vanishing and Exploding Gradient:** Standard RNNs suffer from unstable gradients during backpropagation through time (BPTT). LSTMs mitigate but do not fully eliminate this for very long sequences.
- **Long-Term Dependency Challenges:** LSTMs still struggle when dependencies span extremely long sequences. Transformer-based architectures (e.g., BERT) generally outperform LSTMs in such scenarios.
- **Sequential Processing Bottleneck:** RNNs and LSTMs process data step-by-step, preventing parallelization and increasing training time compared to feedforward or Transformer models.
- **High Parameter Count:** LSTM gates significantly increase the number of parameters compared to simple RNNs, increasing memory usage and overfitting risk on smaller datasets.

## Methodology / Workflow

| Step | Phase | Action |
|---|---|---|
| 1 | Dataset Loading | Load IMDB dataset via tensorflow.keras.datasets with top 10,000 words |
| 2 | Sequence Padding | Pad/truncate all sequences to fixed length of 200 using pad_sequences() |
| 3 | Train-Test Split | 25,000 training reviews and 25,000 testing reviews (built-in split) |
| 4 | Model Architecture | Embedding Layer → LSTM Layer → Dropout → Dense(sigmoid) output |
| 5 | Compilation | Adam optimizer (lr=0.001) with Binary Cross-Entropy loss function |
| 6 | Training | Train for 7 epochs with 20% validation split, batch size 64 |
| 7 | Evaluation | Test accuracy, confusion matrix, prediction probability distribution |

## Hyperparameter Tuning

LSTM-based models are highly sensitive to hyperparameter choices, directly influencing memory capacity, learning stability, and generalization performance. The following hyperparameters were configured for this experiment:

| Hyperparameter | Value | Rationale |
|---|---|---|
| Vocabulary Size (MAX_FEATURES) | 10,000 | Limits vocabulary to frequent words, reducing noise and computation |
| Sequence Length (MAX_LEN) | 200 | Captures sufficient context from most reviews without excess padding |
| Embedding Dimension | 128 | Rich word vector representation capturing semantic relationships |
| LSTM Units | 64 | Balanced trade-off between learning capacity and generalization |
| Dropout Rate | 0.3 | Moderate regularization to reduce overfitting from high LSTM capacity |
| Optimizer | Adam | Adaptive learning rate ensuring efficient and stable convergence |
| Learning Rate | 0.001 | Standard Adam default providing smooth and stable weight updates |
| Epochs | 7 | Sufficient for convergence; further epochs increase overfitting risk |
| Batch Size | 64 | Efficient gradient estimation with manageable memory usage |
| Validation Split | 20% | Reserves 20% of training data to monitor generalization during training |

## Performance Analysis

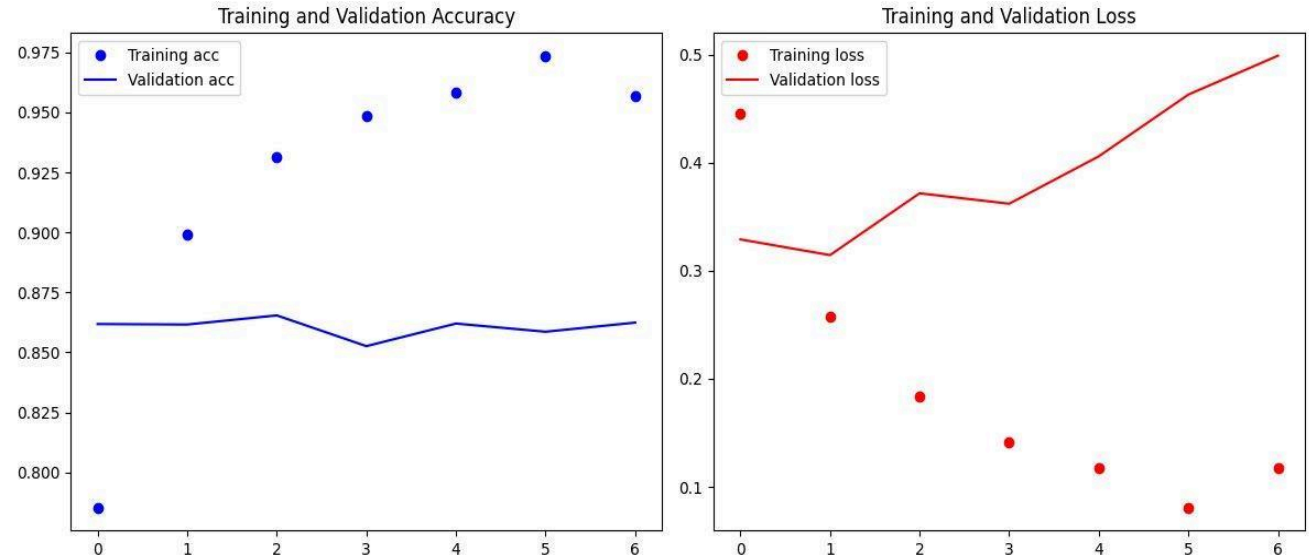
The LSTM model was evaluated on 25,000 unseen IMDB test reviews, achieving a Test Accuracy of 85.19% and a Test Loss of 0.5448.

| Metric | Training (Epoch 7) | Test Set |
|---|---|---|
| Accuracy | ~97.4% | 85.19% |
| Loss | ~0.12 | 0.5448 |

- **Generalization Gap:** Training accuracy reaches ~97% while test accuracy stabilizes at 85.19%, indicating moderate overfitting on longer reviews — a common LSTM characteristic on IMDB without stronger regularization or bidirectional layers.
- **Confusion Matrix:** Out of 25,000 test reviews, 10,998 Negative and 10,299 Positive reviews are correctly classified, with 1,502 false positives and 2,201 false negatives.
- **Prediction Distribution:** The histogram of prediction probabilities shows a strong bimodal distribution, with most predictions confidently near 0 (Negative) or near 1 (Positive), indicating the model makes decisive rather than borderline classifications.

- **Custom Predictions:** The model correctly identifies strongly worded positive and negative reviews but shows some confusion on subtly worded or mixed-sentiment text, reflecting the inherent ambiguity in sentiment analysis.

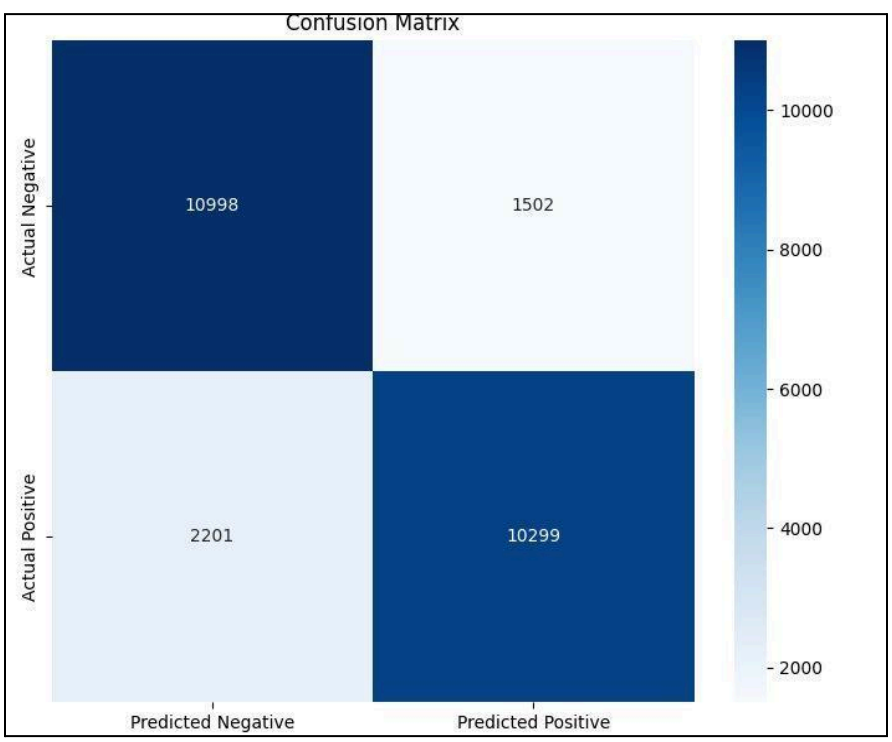
## Output

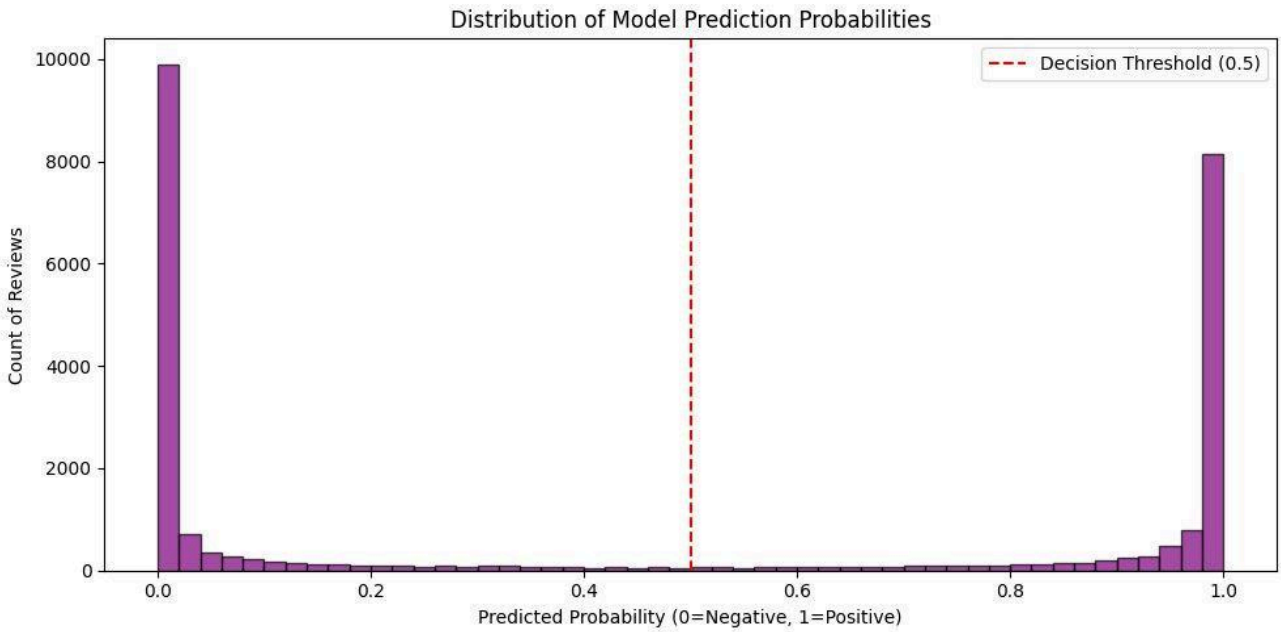



```
Test Accuracy: 85.19%
Test Loss: 0.5448
```

**Test Accuracy:** 85.19%

**Test Loss:** 0.5448






```
1641221/1641221 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Text: 'This movie was absolutely fantastic, loved every moment of i...'
Score: 0.2136
Prediction: NEGATIVE

Text: 'Terrible film, complete waste of time, would not recommend....'
Score: 0.5381
Prediction: POSITIVE

Text: 'The acting was brilliant and the storyline was deeply moving...'
Score: 0.8245
Prediction: POSITIVE

Text: 'Boring and predictable plot with no character development at...'
Score: 0.9218
Prediction: POSITIVE
```

## Conclusion

In this experiment, an LSTM-based Recurrent Neural Network was successfully implemented for binary sentiment classification using the IMDB Movie Reviews dataset. The model achieved a test accuracy of 85.19%, demonstrating effective learning of contextual patterns in sequential text data.

The confusion matrix confirmed reasonable precision and recall across both sentiment classes. The prediction probability distribution showed confident bimodal predictions, indicating the model makes decisive rather than borderline classifications. The training-validation gap suggests moderate overfitting that could be addressed through stronger dropout, early stopping, or Bidirectional LSTM layers.

Overall, this experiment confirms that LSTM networks are well-suited for sequential text modeling and binary sentiment classification, and serve as a strong baseline before exploring Transformer-based architectures.